Design Strategy Pattern
ការងារតែមួយតែ រៀបធ្វើខុសគ្នា

1-Create recode

Recode វាដូចជា Class ដែលតែវាមិនអាចកែប្រែបាន ហើយវាជាកូនចៅរបស់ Class.

```java
public record ShippingFeeRequest(
    String countryCode,
    BigDecimal weightKg,
    BigDecimal orderAmount
) {}
```

បើយើងប្រៀបធៀបទៅនឹង Class គឺវាបង្កើតអោយយើង Assessor auto :

```java
public final class ShippingFeeRequest {
    private final String countryCode;
    private final BigDecimal weightKg;
    private final BigDecimal orderAmount;

    public ShippingFeeRequest(String countryCode, BigDecimal weightKg, BigDecimal ord
        this.countryCode = countryCode;
        this.weightKg = weightKg;
        this.orderAmount = orderAmount;
    }

    public String countryCode() {
        return countryCode;
    }

    public BigDecimal weightKg() {
        return weightKg;
    }

    public BigDecimal orderAmount() {
        return orderAmount;
    }

    @Override
    public boolean equals(Object o) {
        // generated by Java
    }

    @Override
    public int hashCode() {
        // generated by Java
    }

    @Override
    public String toString() {
        // generated by Java
    }
}
```

What is different Recode Vs Class:

A **class** is general-purpose and A **record** is for immutable data carriers.

Why use `record`

Use a record when your object is mainly just data, like:

- request DTO
- response DTO
- input parameters
- value object

Your `ShippingFeeRequest` is a very good use case for `record`.

Limitations of `record`

Compared to a normal class, a record has some restrictions:

- it is implicitly `final`
- fields are implicitly `private final`
- it cannot extend another class
- it is not designed for mutable objects

What a `record` gives you automatically

For each component in your record, Java generates:

- private final fields
- a constructor
- accessor methods
- `equals()`
- `hashCode()`
- `toString()`

Example calculations:

If weight = 0.3 kg
0.3 / 0.5 = 0.6 → round up = 1
Fee = 2000 + (1500 × 1) = 3500

If weight = 0.5 kg
0.5 / 0.5 = 1
Fee = 2000 + (1500 × 1) = 3500

If weight = 0.6 kg
0.6 / 0.5 = 1.2 → round up = 2
Fee = 2000 + (1500 × 2) = 5000

If weight = 1.2 kg
1.2 / 0.5 = 2.4 → round up = 3
Fee = 2000 + (1500 × 3) = 6500

`BigDecimal`

- can store decimals: `123.45`
- very precise
- larger range and flexible scale
- slower than primitive types

`long`

- only whole numbers: `123`
- fixed range: `-9223372036854775808` to `9223372036854775807`
- fast and memory-efficient
- no decimal part

Why conversion can be dangerous

A `BigDecimal` may contain:

1. a fractional part
   Example: `123.99`
2. a value too large for `long`
   Example: `999999999999999999999999`

BigDecimal: លេខមានក្បាស and Long លេខគត់

Note:

`longValue()` returns the value converted to `long`, even if that means losing information.

`longValueExact()` returns the value as a `long` **only if it fits exactly.** Otherwise it throws an exception.

This usually comes up with `BigInteger` and `BigDecimal`.

Example with `BigInteger`:

```java
BigInteger a = new BigInteger("123");
long x = a.longValue();        // 123
long y = a.longValueExact();   // 123
```

But if the value is too large:

```java
BigInteger b = new BigInteger("999999999999999999999999");

long x = b.longValue();        // truncated/wrapped result
long y = b.longValueExact();   // throws ArithmeticException
```

With `BigDecimal`, `longValueExact()` is also stricter about fractional parts:

```java
BigDecimal a = new BigDecimal("42.0");
a.longValue();        // 42
a.longValueExact();   // ok

BigDecimal b = new BigDecimal("42.5");
b.longValue();        // 42
b.longValueExact();   // throws ArithmeticException
```

So the practical rule is:

- Use `longValue()` when you are okay with silent narrowing/truncation.
- Use `longValueExact()` when you want safety and correctness checks.

In most business code, `longValueExact()` is safer when precision matters.

longvalueExact: បើវាទទួលតម្លៃមក លើសចំនួនដែល long ផ្ទុកបានពី fixed range: -9223372036854775808 to 9223372036854775807 វានឹងបង្ហាញ Throw Error

```java
BigDecimal a = new BigDecimal("123.99");
System.out.println(a.longValue());       // 123
System.out.println(a.longValueExact());  // throws ArithmeticException
```

ពេល convert to long វាកាត់ខាងក្បាស និងចោលសម្រាប់ a.longValue() តែបងយើងដាក់ a.longValueExact() វានឹង Throw error អោយយើងដឹង

When to use which

Use `BigDecimal` when:

- working with money
- precision matters
- decimals are allowed

Use `long` when:

- you only need whole numbers
- performance matters
- the target API/type requires `long`

A good rule: keep values as `BigDecimal` as long as precision matters, and convert to `long` only at the boundary where an integer is actually required.

2- Create validation

យើងអាចបង្កើតបានពីប្រភេទ សម្រាប់ពេលហៅវាយកមកប្រើ៖ ពេលហៅដោយមិនចាំបាច់ Create instance and មួយទៀតយើងបង្កើត Object ដើម្បីធ្វើការវាយកមកប្រើ។ អ្វីដែលស្រួលគឺ មិនចាំបាច់បង្កើត Object ដោយធ្វើការ បង្កើត Constructor នៅក្នុង class validation នឹងអោយហើយមុន។

2-1 Create class in Constructor

Java

```java
package com.vibol.java.school;

import java.math.BigDecimal;

public final class ShippingFeeRequestValidator {

    private ShippingFeeRequestValidator() {
        // prevent instantiation
    }

    public static void validate(ShippingFeeRequest request) {
        if (request == null) {
            throw new IllegalArgumentException("Request is required");
        }

        if (request.weightKg() == null) {
            throw new IllegalArgumentException("Weight is required");
        }

        if (request.weightKg().compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Weight must be greater than zero");
        }

        if (request.orderAmount() == null) {
            throw new IllegalArgumentException("Order amount is required");
        }

        if (request.orderAmount().compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Order amount must not be negative");
        }
    }
}
```

⇨ When called in the class:

```java
package com.vibol.java.school;

import java.math.BigDecimal;

public class DefaultShippingFeeStrategy implements ShippingFeeStrategy{

    @Override
    public Long calculate(ShippingFeeRequest request) {

        // Validation
        ShippingFeeRequestValidator.validate(request);

        // Example rules: 5% of order amount, minimum 3000
        long fee = request.orderAmount()
                .multiply(BigDecimal.valueOf(0.05))
                .longValue();

        return Math.max(fee, 3000L);
    }

}
```

2-2 without constructor: we need to create object instance

```java
package com.vibol.java.school;

import java.math.BigDecimal;

public class ShippingFeeRequestValidator {

    public void validate(ShippingFeeRequest request) {
        if (request == null) {
            throw new IllegalArgumentException("Request is required");
        }

        if (request.weightKg() == null) {
            throw new IllegalArgumentException("Weight is required");
        }

        if (request.weightKg().compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Weight must be greater than zero");
        }

        if (request.orderAmount() == null) {
            throw new IllegalArgumentException("Order amount is required");
        }

        if (request.orderAmount().compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Order amount must not be negative");
        }
    }
}
```

⇨ When called in the class

```java
package com.vibol.java.school;

import java.math.BigDecimal;

public class DefaultShippingFeeStrategy implements ShippingFeeStrategy {

    private final ShippingFeeRequestValidator validator = new ShippingFeeRequestValic

    @Override
    public Long calculate(ShippingFeeRequest request) {
        validator.validate(request);

        long fee = request.orderAmount()
                .multiply(BigDecimal.valueOf(0.05))
                .longValue();

        return Math.max(fee, 3000L);
    }
}
```

3- Create Service:

```java
package com.vibol.java.school.service;

import java.util.Map;

import com.vibol.java.school.ShippingFeeStrategy;

public final class ShippingFeeService {

    private final Map<String, ShippingFeeStrategy> strategies;
    private final ShippingFeeStrategy defaultStrategy;

    public ShippingFeeService(
            Map<String, ShippingFeeStrategy> strategies,
            ShippingFeeStrategy defaultStrategy) {
        this.strategies = strategies;
        this.defaultStrategy = defaultStrategy;
    }
}
```

private final Map<String, ShippingFeeStrategy> strategies;

យើងបង្កើត Filed វាជា Mapping ដែលមាន Key and Value. Key = “kh” ចឹងវាទៅរក មើល implement from the Interface: ShippingFeeStrategy តើវាត្រូវមួយណាចឹងវា នឹងយក Implement class នឹងមកប្រើ

4- Create service

```
package com.vibol.java.school.service;

import java.util.Map;

public final class ShippingFeeService {

    private final Map<String, ShippingFeeStrategy> strategies;
    private final ShippingFeeStrategy defaultStrategy;

    //Constractor
    public ShippingFeeService(

            Map<String, ShippingFeeStrategy> strategies,

            ShippingFeeStrategy defaultStrategy) {

        this.strategies = strategies;
        this.defaultStrategy = defaultStrategy;

    }

    public long calculate(ShippingFeeRequest request) {

        ShippingFeeStrategy strategy = strategies.getOrDefault(request.countryCode(),
                defaultStrategy);

        return strategy.calculate(request);

    }
}
```

យើងបង្កើត Service ដែលមានពី Field input :

1- Strategies: សម្រាប់ថាយើងចង់ប្រើប្រាស់ Implement ប្រទេសមួយណា

2- defaultStrategy: សម្រាប់ប្រើប្រាស់ អ្វីដែលក្រៅពីមាននៅក្នុង implementation

3- បង្កើត Method calculate ដែលទទួល Request ចូលមក

4- Running the calculation

```java
import com.vibol.java.school.service.ShippingFeeService;

public class Demo {
    public static void main(String[] args) {
        Map<String, ShippingFeeStrategy> map = getStrategy();
        ShippingFeeService service = new ShippingFeeService(
                map,
                new DefaultShippingFeeStrategy()
        );
        /*
         In our ShippingFeeService need two parameter:
            1- Map<String, ShippingFeeStrategy> strategies;
            2- ShippingFeeStrategy defaultStrategy;
         */
        String countryCode = "KH";
        BigDecimal weightKg = new BigDecimal("2.1");
        BigDecimal orderAmount = new BigDecimal("10000");

        ShippingFeeRequest request = new ShippingFeeRequest(countryCode, weightKg, orderAmount);
        long fee = service.calculate(request);

        System.out.println("Fee = " + fee);
    }

    //our function create
    private static Map<String, ShippingFeeStrategy>getStrategy(){

        //Create new object instance
        Map<String, ShippingFeeStrategy> map = new HashMap<>();

        map.put("KH", new CambodiaShippingFeeStrategy());
        map.put("LA", new LaoShippingFeeStrategy());

        return map;
    }
```

Why we use new to create “

BigDecimal weightKg = new BigDecimal("2.1");

”.

`new` is needed there because `BigDecimal` is a class, so you are creating a new object instance.

In this part:

```java
String countryCode = "KH";
BigDecimal weightKg = new BigDecimal("2.1");
BigDecimal orderAmount = new BigDecimal("10000");

ShippingFeeRequest request = new ShippingFeeRequest(countryCode, weightKg, orderAmour
```

there are actually **two different things happening**:

1. Creating `BigDecimal` objects

```java
BigDecimal weightKg = new BigDecimal("2.1");
BigDecimal orderAmount = new BigDecimal("10000");
```

`BigDecimal` is not a primitive type like `int`, `double`, or `long`.
It is a Java class from `java.math`.

So to make a `BigDecimal`, Java needs:

```java
new BigDecimal("2.1")
```

That means:

- allocate memory for a new `BigDecimal` object
- call its constructor
- store the reference in `weightKg`